

Nama : Jovita Fashya Islami

Nim : H1D024125

Shift Lama : G

Shift Baru : F

TUGAS PRAKTIKUM KECERDASAN BUATAN PERTEMUAN 7

Step 1 - Import Library

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
```

Library yang digunakan meliputi tensorflow sebagai framework utama deep learning, Sequential dan Dense dari Keras untuk membangun arsitektur model, pandas untuk membaca dan mengolah dataset, numpy untuk komputasi array, LabelEncoder untuk mengonversi label kategorikal menjadi numerik, serta train_test_split untuk membagi dataset menjadi data latih dan data uji.

Step 2 - Load Dataset

```
# Muat dataset iris dari file CSV/data
dataset = pd.read_csv('iris.data', header=None, sep=',')

# Menyusun data X (fitur) dan y (label)
X = dataset.iloc[:, :-1].values # 4 kolom pertama sebagai fitur
y = dataset.iloc[:, -1].values  # Kolom terakhir sebagai label
```

Dataset Iris dimuat dari file lokal iris.data menggunakan fungsi pd.read_csv dengan parameter header=None karena file tidak memiliki baris judul kolom, dan sep=',' sebagai pemisah antar nilai. Empat kolom pertama disimpan sebagai variabel X yang merepresentasikan fitur, sedangkan kolom terakhir disimpan sebagai variabel y yang merepresentasikan label kelas spesies bunga.

Step 3 - Konversi Label

```
# Mengonversi label dari string menjadi numerik
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y) # Mengubah label jadi 0, 1, 2
```

Label kelas berupa string dikonversi menjadi nilai numerik menggunakan LabelEncoder. Hasil konversi menghasilkan pemetaan Iris-setosa menjadi 0, Iris-versicolor menjadi 1, dan Iris-virginica menjadi 2 agar dapat diproses oleh model.

Step 4 - Split Data Training dan Testing

```
# Memisahkan dataset menjadi data latih dan data validasi dengan rasio 80:20
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Dataset dibagi dengan rasio 80:20 menggunakan train_test_split, menghasilkan 120 data latih dan 30 data uji. Parameter random_state=42 digunakan agar pembagian data bersifat konsisten setiap kali program dijalankan.

Step 5 - Membuat Arsitektur Model

```
model = Sequential([
    Input(shape=X_train.shape[1:]),
    Dense(1000, activation='relu'),
    Dense(500, activation='relu'),
    Dense(300, activation='relu'),
    Dense(3, activation='softmax')
])

model.summary()
```

Model dibangun menggunakan Sequential dengan susunan layer Input yang menerima 4 fitur, tiga Dense layer tersembunyi dengan masing-masing 1000, 500, dan 300 neuron menggunakan fungsi aktivasi ReLU, serta satu Dense layer output dengan 3 neuron dan fungsi aktivasi Softmax sesuai jumlah kelas. Fungsi model.summary() digunakan untuk menampilkan ringkasan arsitektur model.

Step 6 - Compile Model

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Model dikompilasi dengan optimizer Adam yang adaptif dan efisien, fungsi loss sparse_categorical_crossentropy yang sesuai untuk klasifikasi multikelas dengan label integer, serta metrics accuracy untuk memantau performa model selama proses pelatihan.

Step 7 - Training Model

```
history = model.fit(
    X_train, y_train,
    epochs=50,
```

```
batch_size=32,  
validation_data=(X_test, y_test)  
)
```

Proses pelatihan dilakukan selama 50 epoch dengan batch_size sebesar 32. Parameter validation_data digunakan untuk mengevaluasi model pada data uji di setiap epoch guna memantau kemungkinan terjadinya overfitting. Hasil pelatihan disimpan dalam variabel history untuk keperluan visualisasi.

Step 8 - Evaluasi Model

```
loss, accuracy = model.evaluate(X_test, y_test)  
print(f"Loss: {loss}, Accuracy: {accuracy}")
```

Model dievaluasi menggunakan data uji melalui fungsi model.evaluate yang menghasilkan nilai loss dan accuracy akhir. Hasil evaluasi menunjukkan accuracy sebesar 96.67% dengan nilai loss sebesar 0.0672, yang berarti model berhasil mengklasifikasikan 29 dari 30 data uji dengan benar.

Step 9 - Visualisasi Training History

```
pd.DataFrame(history.history).plot(figsize=(10,6))
```

Nilai loss, accuracy, val_loss, dan val_accuracy yang tersimpan dalam history.history divisualisasikan menggunakan fungsi plot dari pandas DataFrame. Grafik menunjukkan bahwa nilai loss menurun secara signifikan di awal epoch kemudian stabil, sementara accuracy meningkat dan konsisten di kisaran 95-97%.

Step 10 - Prediksi Data Testing

```
predictions = model.predict(X_test)  
# Mengambil indeks dari nilai probabilitas tertinggi untuk setiap prediksi  
predicted_classes = predictions.argmax(axis=1)  
print("Prediksi:", predicted_classes)  
print("Label Asli:", y_test)
```

Fungsi model.predict digunakan untuk menghasilkan nilai probabilitas setiap kelas pada data uji. Fungsi argmax(axis=1) kemudian digunakan untuk mengambil indeks dengan probabilitas tertinggi sebagai kelas prediksi, yang selanjutnya dibandingkan dengan label asli.

Step 11 - Confusion Matrix

```
from sklearn.metrics import confusion_matrix  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# Buat confusion matrix  
cm = confusion_matrix(y_test, predicted_classes)  
# Visualisasikan confusion matrix
```

```
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```

Confusion matrix dibuat menggunakan fungsi `confusion_matrix` dari `sklearn` untuk membandingkan label asli dengan hasil prediksi secara detail per kelas. Visualisasi menggunakan `sns.heatmap` menampilkan matriks dalam bentuk heatmap berwarna. Hasil menunjukkan Iris-setosa 10/10 benar, Iris-versicolor 8/9 benar, dan Iris-virginica 11/11 benar.

Step 12 - Prediksi Data Baru

```
# Fungsi untuk memprediksi data input baru
def predict_new_data():
    sepal_length = float(input("Masukkan sepal length: "))
    sepal_width = float(input("Masukkan sepal width: "))
    petal_length = float(input("Masukkan petal length: "))
    petal_width = float(input("Masukkan petal width: "))

    # Membuat data array baru
    new_data = np.array([[sepal_length, sepal_width, petal_length,
                           petal_width]])

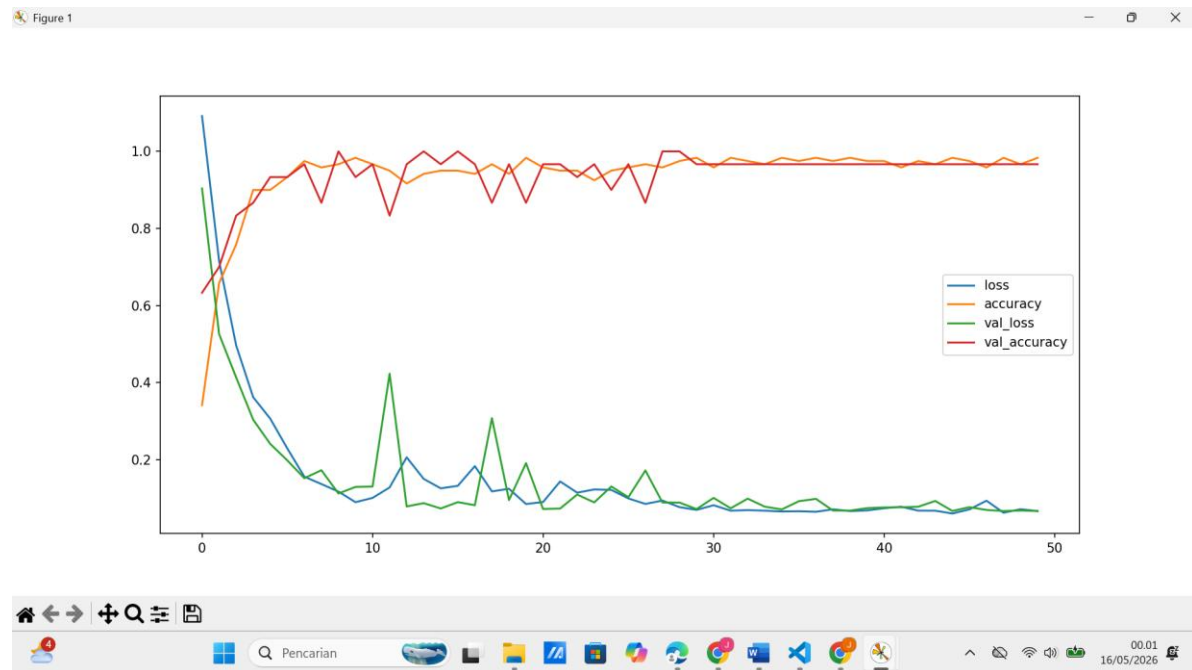
    # Melakukan prediksi
    prediction = model.predict(new_data)
    predicted_class = prediction.argmax(axis=1)

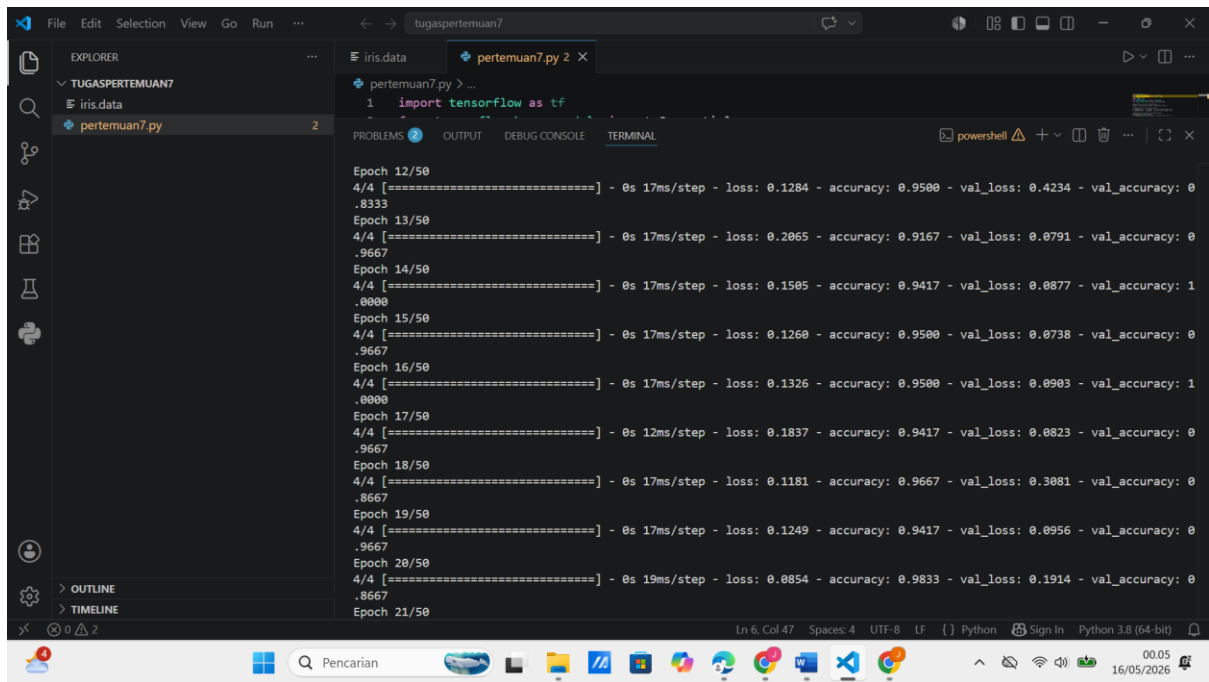
    # Mengonversi hasil prediksi numerik menjadi label asli
    predicted_label = label_encoder.inverse_transform(predicted_class)
    print(f"Prediksi kelas: {predicted_label[0]}")

predict_new_data()
```

Fungsi `predict_new_data` dibuat untuk menerima input data baru dari pengguna berupa nilai sepal length, sepal width, petal length, dan petal width. Data tersebut dikonversi menjadi array numpy lalu diproses oleh model untuk menghasilkan prediksi kelas. Hasil prediksi numerik dikembalikan ke nama spesies asli menggunakan fungsi `inverse_transform` dari `label_encoder`.

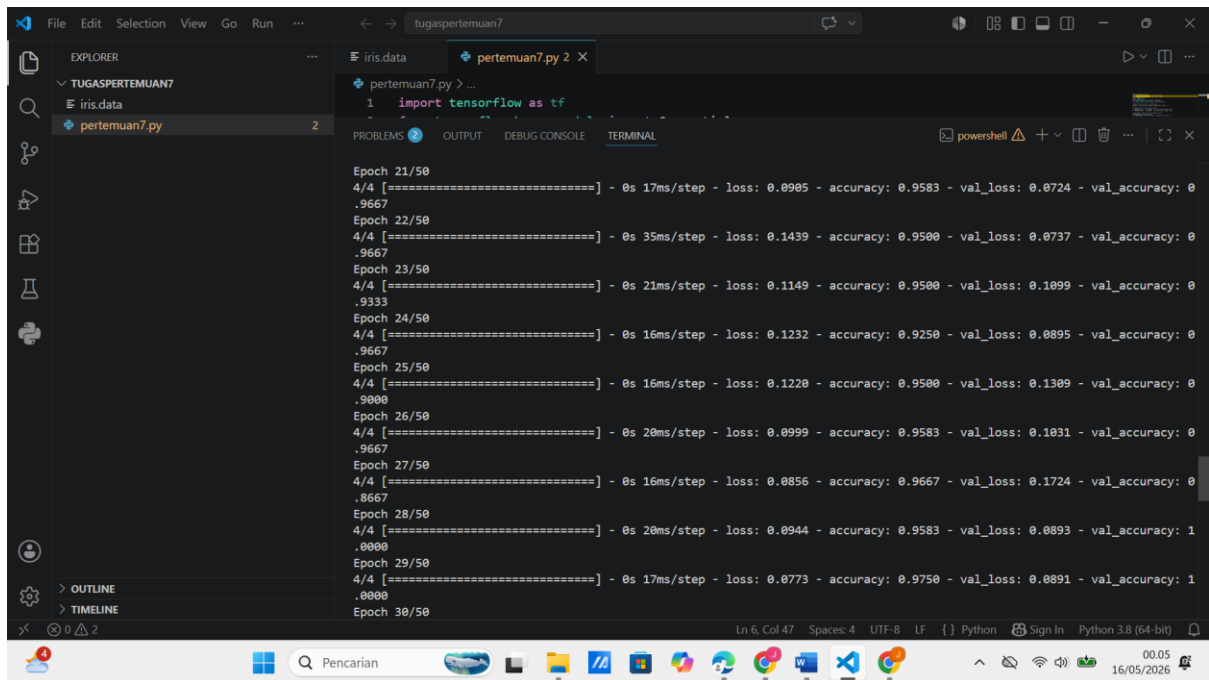
Hasil Output





```
File Edit Selection View Go Run ...
tugaspertemuan7
EXPLORER
TUGASPERTEMUAN7
  iris.data
  pertemuan7.py
pertemuan7.py
1 import tensorflow as tf

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
Epoch 12/50
4/4 [=====] - 0s 17ms/step - loss: 0.1284 - accuracy: 0.9500 - val_loss: 0.4234 - val_accuracy: 0.8333
Epoch 13/50
4/4 [=====] - 0s 17ms/step - loss: 0.2065 - accuracy: 0.9167 - val_loss: 0.0791 - val_accuracy: 0.9667
Epoch 14/50
4/4 [=====] - 0s 17ms/step - loss: 0.1505 - accuracy: 0.9417 - val_loss: 0.0877 - val_accuracy: 1.0000
Epoch 15/50
4/4 [=====] - 0s 17ms/step - loss: 0.1260 - accuracy: 0.9500 - val_loss: 0.0738 - val_accuracy: 0.9667
Epoch 16/50
4/4 [=====] - 0s 17ms/step - loss: 0.1326 - accuracy: 0.9500 - val_loss: 0.0903 - val_accuracy: 1.0000
Epoch 17/50
4/4 [=====] - 0s 12ms/step - loss: 0.1837 - accuracy: 0.9417 - val_loss: 0.0823 - val_accuracy: 0.9667
Epoch 18/50
4/4 [=====] - 0s 17ms/step - loss: 0.1181 - accuracy: 0.9667 - val_loss: 0.3081 - val_accuracy: 0.8667
Epoch 19/50
4/4 [=====] - 0s 17ms/step - loss: 0.1249 - accuracy: 0.9417 - val_loss: 0.0956 - val_accuracy: 0.9667
Epoch 20/50
4/4 [=====] - 0s 19ms/step - loss: 0.0854 - accuracy: 0.9833 - val_loss: 0.1914 - val_accuracy: 0.8667
Epoch 21/50
Ln 6, Col 47 Spaces: 4 UTF-8 LF Python Sign In Python 3.8 (64-bit)
Pencarian 00.05 16/05/2026
```



```
Epoch 21/50
4/4 [=====] - 0s 17ms/step - loss: 0.0905 - accuracy: 0.9583 - val_loss: 0.0724 - val_accuracy: 0.9667
Epoch 22/50
4/4 [=====] - 0s 35ms/step - loss: 0.1439 - accuracy: 0.9500 - val_loss: 0.0737 - val_accuracy: 0.9667
Epoch 23/50
4/4 [=====] - 0s 21ms/step - loss: 0.1149 - accuracy: 0.9500 - val_loss: 0.1099 - val_accuracy: 0.9333
Epoch 24/50
4/4 [=====] - 0s 16ms/step - loss: 0.1232 - accuracy: 0.9250 - val_loss: 0.0895 - val_accuracy: 0.9667
Epoch 25/50
4/4 [=====] - 0s 16ms/step - loss: 0.1220 - accuracy: 0.9500 - val_loss: 0.1309 - val_accuracy: 0.9000
Epoch 26/50
4/4 [=====] - 0s 20ms/step - loss: 0.0999 - accuracy: 0.9583 - val_loss: 0.1831 - val_accuracy: 0.9667
Epoch 27/50
4/4 [=====] - 0s 16ms/step - loss: 0.0856 - accuracy: 0.9667 - val_loss: 0.1724 - val_accuracy: 0.8667
Epoch 28/50
4/4 [=====] - 0s 20ms/step - loss: 0.0944 - accuracy: 0.9583 - val_loss: 0.0893 - val_accuracy: 1.0000
Epoch 29/50
4/4 [=====] - 0s 17ms/step - loss: 0.0773 - accuracy: 0.9750 - val_loss: 0.0891 - val_accuracy: 1.0000
Epoch 30/50
Ln 6, Col 47 Spaces: 4 UTF-8 LF Python Sign In Python 3.8 (64-bit)
Pencarian 00.05 16/05/2026
```

The screenshot shows the Visual Studio Code interface with a file explorer on the left containing 'TUGASPERTEMUAN7', 'iris.data', and 'pertemuan7.py'. The main editor displays the 'pertemuan7.py' file with the first line being `import tensorflow as tf`. The terminal window at the bottom shows the output of a training process for epochs 30 through 39. Each epoch's output includes a progress bar (4/4), time per step (0s 17ms/step), loss, accuracy, validation loss, and validation accuracy. The loss and accuracy values fluctuate slightly across epochs, while validation loss and accuracy remain at 0.

```
Epoch 30/50
4/4 [=====] - 0s 17ms/step - loss: 0.0761 - accuracy: 0.9833 - val_loss: 0.0723 - val_accuracy: 0
.9667
Epoch 31/50
4/4 [=====] - 0s 17ms/step - loss: 0.0823 - accuracy: 0.9583 - val_loss: 0.1013 - val_accuracy: 0
.9667
Epoch 32/50
4/4 [=====] - 0s 16ms/step - loss: 0.0681 - accuracy: 0.9833 - val_loss: 0.0743 - val_accuracy: 0
.9667
Epoch 33/50
4/4 [=====] - 0s 17ms/step - loss: 0.0697 - accuracy: 0.9750 - val_loss: 0.0994 - val_accuracy: 0
.9667
Epoch 34/50
4/4 [=====] - 0s 17ms/step - loss: 0.0681 - accuracy: 0.9667 - val_loss: 0.0785 - val_accuracy: 0
.9667
Epoch 35/50
4/4 [=====] - 0s 17ms/step - loss: 0.0661 - accuracy: 0.9833 - val_loss: 0.0714 - val_accuracy: 0
.9667
Epoch 36/50
4/4 [=====] - 0s 17ms/step - loss: 0.0667 - accuracy: 0.9750 - val_loss: 0.0927 - val_accuracy: 0
.9667
Epoch 37/50
4/4 [=====] - 0s 17ms/step - loss: 0.0653 - accuracy: 0.9833 - val_loss: 0.0988 - val_accuracy: 0
.9667
Epoch 38/50
4/4 [=====] - 0s 17ms/step - loss: 0.0717 - accuracy: 0.9750 - val_loss: 0.0685 - val_accuracy: 0
.9667
Epoch 39/50
```

This screenshot continues the training process from the previous one, showing epochs 39 through 48. The output format remains consistent, displaying epoch number, progress, time per step, loss, accuracy, validation loss, and validation accuracy. The training metrics continue to fluctuate, with validation loss and accuracy consistently at 0.

```
Epoch 39/50
4/4 [=====] - 0s 18ms/step - loss: 0.0670 - accuracy: 0.9833 - val_loss: 0.0683 - val_accuracy: 0
.9667
Epoch 40/50
4/4 [=====] - 0s 17ms/step - loss: 0.0687 - accuracy: 0.9750 - val_loss: 0.0748 - val_accuracy: 0
.9667
Epoch 41/50
4/4 [=====] - 0s 17ms/step - loss: 0.0744 - accuracy: 0.9750 - val_loss: 0.0765 - val_accuracy: 0
.9667
Epoch 42/50
4/4 [=====] - 0s 17ms/step - loss: 0.0785 - accuracy: 0.9583 - val_loss: 0.0770 - val_accuracy: 0
.9667
Epoch 43/50
4/4 [=====] - 0s 15ms/step - loss: 0.0683 - accuracy: 0.9750 - val_loss: 0.0784 - val_accuracy: 0
.9667
Epoch 44/50
4/4 [=====] - 0s 16ms/step - loss: 0.0680 - accuracy: 0.9667 - val_loss: 0.0933 - val_accuracy: 0
.9667
Epoch 45/50
4/4 [=====] - 0s 19ms/step - loss: 0.0608 - accuracy: 0.9833 - val_loss: 0.0673 - val_accuracy: 0
.9667
Epoch 46/50
4/4 [=====] - 0s 16ms/step - loss: 0.0715 - accuracy: 0.9750 - val_loss: 0.0773 - val_accuracy: 0
.9667
Epoch 47/50
4/4 [=====] - 0s 17ms/step - loss: 0.0940 - accuracy: 0.9583 - val_loss: 0.0700 - val_accuracy: 0
.9667
Epoch 48/50
```


The image shows a Visual Studio Code editor window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'TUGASPERTEMUAN7' containing files 'iris.data' and 'pertemuan7.py'. The 'pertemuan7.py' file is open in the editor, showing a Python script that imports TensorFlow and Keras, defines a neural network model, and trains it on the Iris dataset. The terminal window shows the output of the script, including training progress for epochs 46 to 50, and a prediction on a new sample.

```
1 import tensorflow as tf
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Dense
4 from tensorflow.keras.optimizers import Adam
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.datasets import load_iris
7 from sklearn.model_selection import train_test_split
8
9 # Load the Iris dataset
10 iris = load_iris()
11 X = iris.data[:, :4]  # Features
12 y = iris.target        # Target
13
14 # Split the data into training and testing sets
15 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
16
17 # Standardize the features
18 scaler = StandardScaler()
19 X_train = scaler.fit_transform(X_train)
20 X_test = scaler.transform(X_test)
21
22 # Build the neural network model
23 model = Sequential([
24     Dense(4, activation='relu'),
25     Dense(4, activation='relu'),
26     Dense(3, activation='softmax')
27 ])
28 model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy'])
29
30 # Train the model
31 model.fit(X_train, y_train, epochs=50, batch_size=10, validation_data=(X_test, y_test))
32
33 # Predict on a new sample
34 new_sample = [[5.1, 4.9, 5.5, 2.5]]
35 new_sample = scaler.transform(new_sample)
36 prediction = model.predict(new_sample)
37 print("Prediksi kelas: ", iris.target_names[prediction[0]])
```

Terminal Output:

```
.9667
Epoch 46/50
4/4 [=====] - 0s 16ms/step - loss: 0.0715 - accuracy: 0.9750 - val_loss: 0.0773 - val_accuracy: 0
.9667
Epoch 47/50
4/4 [=====] - 0s 17ms/step - loss: 0.0940 - accuracy: 0.9583 - val_loss: 0.0700 - val_accuracy: 0
.9667
Epoch 48/50
4/4 [=====] - 0s 17ms/step - loss: 0.0628 - accuracy: 0.9833 - val_loss: 0.0672 - val_accuracy: 0
.9667
Epoch 49/50
4/4 [=====] - 0s 16ms/step - loss: 0.0719 - accuracy: 0.9667 - val_loss: 0.0685 - val_accuracy: 0
.9667
Epoch 50/50
4/4 [=====] - 0s 17ms/step - loss: 0.0671 - accuracy: 0.9833 - val_loss: 0.0672 - val_accuracy: 0
.9667
1/1 [=====] - 0s 29ms/step - loss: 0.0672 - accuracy: 0.9667
Loss: 0.06724725663661957, Accuracy: 0.9666666388511658
1/1 [=====] - 0s 47ms/step
Prediksi: [1 0 2 1 1 0 1 2 2 1 2 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Label Asli: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Masukkan sepal length: 5.1
Masukkan sepal width: 2.3
Masukkan petal length: 4.1
Masukkan petal width: 2.5
1/1 [=====] - 0s 15ms/step
Prediksi kelas: Iris-virginica
PS C:\Users\jovita fashya\Downloads\Praktikum kb\tugaspertemuan7>
```